

HDOC DAY – 32

ALGORITHM:

1. Start.
2. Display the menu:
 - Matrix addition – prove commutative property.
 - Matrix multiplication – prove it is not commutative in general.
 - Exit.
3. Read the user's choice.
4. Use switch:
 - Choice 1: Call matrixAddition().
 - Read rows and columns.
 - Declare two VLAs of the same size.
 - Read matrices A and B.
 - Calculate $A + B$ and $B + A$.
 - Compare both results.
 - If equal, show that matrix addition is commutative.
 - Choice 2: Call matrixMultiplication().
 - For both AB and BA to exist and have the same dimensions for direct comparison, use two square matrices of order n .
 - Read matrices A and B.
 - Calculate AB and BA .
 - Compare them.
 - If different, the entered example demonstrates that matrix multiplication is not commutative.
 - If equal, explain that these particular matrices commute, although multiplication is not commutative in general.
 - Choice 3: Exit.
 - Otherwise display an invalid-choice message.

5. Repeat until Exit is selected.

6. Stop.

C PROGRAM:

```
#include <stdio.h>
```

```
// Function declarations
```

```
void matrixAddition(void);
```

```
void matrixMultiplication(void);
```

```
int main()
```

```
{
```

```
    int choice;
```

```
    do
```

```
    {
```

```
        printf("\n----- MATRIX OPERATIONS ----- \n");
```

```
        printf("1. Prove matrix addition is commutative\n");
```

```
        printf("2. Prove matrix multiplication is not commutative\n");
```

```
        printf("3. Exit\n");
```

```
        printf("Enter your choice: ");
```

```
        scanf("%d", &choice);
```

```
        switch (choice)
```

```
        {
```

```
            case 1:
```

```
                matrixAddition();
```

```
                break;
```

case 2:

matrixMultiplication();

break;

case 3:

printf("Exiting the program...\n");

break;

default:

printf("Invalid choice! Please enter 1, 2 or 3.\n");

}

} while (choice != 3);

return 0;

}

// Function to prove $A + B = B + A$

void matrixAddition(void)

{

int rows, cols, i, j;

printf("\nEnter number of rows and columns: ");

scanf("%d %d", &rows, &cols);

if (rows <= 0 || cols <= 0)

```
{  
    printf("Rows and columns must be positive.\n");  
    return;  
}
```

```
// Variable Length Arrays
```

```
int A[rows][cols];
```

```
int B[rows][cols];
```

```
int sumAB[rows][cols];
```

```
int sumBA[rows][cols];
```

```
printf("\nEnter elements of Matrix A:\n");
```

```
for (i = 0; i < rows; i++)
```

```
{
```

```
    for (j = 0; j < cols; j++)
```

```
    {
```

```
        scanf("%d", &A[i][j]);
```

```
    }
```

```
}
```

```
printf("\nEnter elements of Matrix B:\n");
```

```
for (i = 0; i < rows; i++)
```

```
{
```

```
    for (j = 0; j < cols; j++)
```

```
    {
```

```
        scanf("%d", &B[i][j]);
```

```
    }
```

```
}
```

```
// Calculate A + B and B + A
for (i = 0; i < rows; i++)
{
    for (j = 0; j < cols; j++)
    {
        sumAB[i][j] = A[i][j] + B[i][j];
        sumBA[i][j] = B[i][j] + A[i][j];
    }
}
```

```
printf("\nA + B:\n");
for (i = 0; i < rows; i++)
{
    for (j = 0; j < cols; j++)
    {
        printf("%d\t", sumAB[i][j]);
    }
    printf("\n");
}
```

```
printf("\nB + A:\n");
for (i = 0; i < rows; i++)
{
    for (j = 0; j < cols; j++)
    {
        printf("%d\t", sumBA[i][j]);
    }
}
```

```

        printf("\n");
    }

    // Compare A+B and B+A
    int equal = 1;

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            if (sumAB[i][j] != sumBA[i][j])
            {
                equal = 0;
            }
        }
    }

    if (equal)
    {
        printf("\nA + B = B + A\n");
        printf("Hence, matrix addition is commutative.\n");
    }
}

```

```

// Function to demonstrate AB != BA
void matrixMultiplication(void)
{

```

```
int n, i, j, k;
```

```
printf("\nEnter the order of the square matrices: ");
```

```
scanf("%d", &n);
```

```
if (n <= 0)
```

```
{
```

```
    printf("Order must be positive.\n");
```

```
    return;
```

```
}
```

```
// Variable Length Arrays
```

```
int A[n][n];
```

```
int B[n][n];
```

```
int AB[n][n];
```

```
int BA[n][n];
```

```
printf("\nEnter elements of Matrix A:\n");
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
    for (j = 0; j < n; j++)
```

```
    {
```

```
        scanf("%d", &A[i][j]);
```

```
    }
```

```
}
```

```
printf("\nEnter elements of Matrix B:\n");
```

```
for (i = 0; i < n; i++)
```

```
{  
    for (j = 0; j < n; j++)  
    {  
        scanf("%d", &B[i][j]);  
    }  
}
```

```
// Initialize AB and BA to 0
```

```
for (i = 0; i < n; i++)  
{  
    for (j = 0; j < n; j++)  
    {  
        AB[i][j] = 0;  
        BA[i][j] = 0;  
    }  
}
```

```
// Calculate AB
```

```
for (i = 0; i < n; i++)  
{  
    for (j = 0; j < n; j++)  
    {  
        for (k = 0; k < n; k++)  
        {  
            AB[i][j] += A[i][k] * B[k][j];  
        }  
    }  
}
```

```
// Calculate BA
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        for (k = 0; k < n; k++)
        {
            BA[i][j] += B[i][k] * A[k][j];
        }
    }
}
```

```
printf("\nA x B:\n");
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        printf("%d\t", AB[i][j]);
    }
    printf("\n");
}
```

```
printf("\nB x A:\n");
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
```

```
        printf("%d\t", BA[i][j]);  
    }  
    printf("\n");  
}
```

```
// Compare AB and BA
```

```
int equal = 1;
```

```
for (i = 0; i < n; i++)  
{  
    for (j = 0; j < n; j++)  
    {  
        if (AB[i][j] != BA[i][j])  
        {  
            equal = 0;  
        }  
    }  
}
```

```
if (equal)
```

```
{  
    printf("\nFor these particular matrices, AB = BA.\n");  
    printf("These matrices commute, but matrix multiplication ");  
    printf("is not commutative in general.\n");  
}
```

```
else
```

```
{  
    printf("\nAB != BA\n");  
}
```

```

printf("Hence, this example demonstrates that matrix ");

printf("multiplication is not commutative.\n");

}

}

```

TEST CASE TABLE:

TEST CASE	CHOICE	INPUT	EXPECTED RESULT
1	1	2 x 2, A = 1 2; 3 4, B = 5 6; 7 8	A+B = B+A
2	2	2 x 2, A = 1 2; 3 4, B = 5 6; 7 8	AB != BA
3	2	Identity matrix and any 2 x 2 matrix	AB = BA for this particular pair
4	1	Rows = 0	Error: dimensions must be positive
5	2	Order = -2	Error: order must be positive
6	4	--	Invalid choice
7	3	--	Program exits